

CAP THEOREM

DOSEN PENGAMPU: RUNAL REZKIAWAN, S.Kom., M.T



Disusun Oleh:

NADILLA GETALIA

105841106623

PROGRAM STUDI INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH MAKASSAR

2026

1. Materi Singkat

a. Sistem Terdistribusi

Sistem terdistribusi adalah sekumpulan komputer mandiri (node) yang terhubung melalui jaringan dan saling berkoordinasi menggunakan pertukaran pesan (message passing) sehingga terlihat oleh pengguna akhir sebagai satu kesatuan sistem tunggal yang utuh. Karakteristik utamanya meliputi pengerjaan komputasi secara paralel (concurrency), tidak adanya acuan waktu universal (no global clock), serta kegagalan komponen yang bersifat independen (independent failures).

b. Konsep Teorema CAP (CAP Theorem)

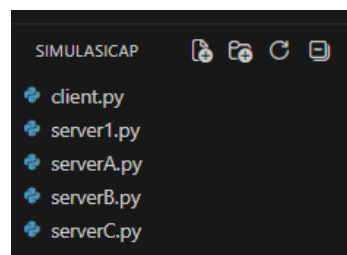
Dikemukakan oleh Eric Brewer, Teorema CAP menyatakan bahwa sebuah sistem data terdistribusi yang berjalan di atas jaringan internet fisik mustahil dapat menjamin tiga properti berikut secara bersamaan secara 100%:

- Consistency (Konsistensi - C): Setiap operasi baca (read) menjamin kembalinya data paling mutakhir dari operasi tulis (write) terakhir di node manapun. Seluruh node melihat status data yang sama pada waktu yang sama.
- Availability (Ketersediaan - A): Setiap request dari node yang tidak mengalami error/kegagalan wajib menerima respons non-error, tanpa ada jaminan bahwa data tersebut adalah versi paling mutakhir (stale data diperbolehkan).
- Partition Tolerance (Toleransi Partisi - P): Sistem tetap mampu beroperasi dan berfungsi dengan baik meskipun terjadi pemutusan atau gangguan komunikasi (network partition) antar-node di dalam jaringan.

Trade-off Fundamental: Karena jaringan fisik di dunia nyata pasti berisiko mengalami gangguan (P tidak bisa dihindari), arsitek sistem harus memilih trade-off mutlak antara menjaga konsistensi data penuh (CP System) atau menjaga ketersediaan layanan tanpa interupsi (AP System).

2. Langkah-langkah Penyelesaian Program

a. Isi File Program



- serverA.py / serverB.py / serverC.py (Node Cluster Backend): Berperan sebagai node penyimpan data lokal (variabel saldo). Masing-masing dikonfigurasi berjalan di port unik lokalserver (5001, 5002, 5003) untuk mendengarkan instruksi baca (READ) dan tulis (WRITE).
- server1.py (Gateway / Central Coordinator): Bertindak sebagai penengah (proxy) yang menerima request pertama kali dari client. File ini memegang kendali simulasi status jaringan (network_status = True/False). Di sinilah aturan penolakan transaksi (Consistency Guard) diimplementasikan apabila terdeteksi status jaringan ke salah satu node terputus (network partition).
- client.py (Aplikasi Penguji): Menjalankan sekuensial instruksi pengujian otomatis untuk melakukan Read awal saldo di seluruh node, melakukan Write (update) saldo saat kondisi jaringan normal, serta melakukan Write kembali setelah partisi jaringan terjadi untuk menguji ketahanan Teorema CAP.

b. Eksekusi Program Di VS Code

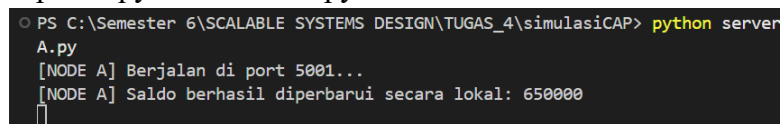
Untuk menjalankan simulasi secara utuh, buka 5 panel Terminal terpisah di VS Code, lalu jalankan perintah berikut secara berurutan:

- Terminal 1: python serverA.py
- Terminal 2: python serverB.py
- Terminal 3: python serverC.py
- Terminal 4: python server1.py
- Terminal 5: python client.py

c. Hasil Pengujian

Halaman ini menampilkan log keluaran (output) dari aplikasi simulasi terdistribusi setelah dijalankan melalui terminal VS Code. Pengujian dibagi ke dalam dua skenario utama untuk membuktikan implementasi model CP (Consistency + Partition Tolerance):

- Output 1: python serverA.py



```

PS C:\Semester 6\SCALABLE SYSTEMS DESIGN\TUGAS_4\simulasiCAP> python serverA.py
[NODE A] Berjalan di port 5001...
[NODE A] Saldo berhasil diperbarui secara lokal: 650000

```

Terminal ini menjalankan skrip serverA.py yang bertindak sebagai Node A di dalam klaster terdistribusi. Server ini dikonfigurasi menggunakan port 5001. Saat pertama kali dijalankan, muncul log [NODE A] Berjalan di port 5001....

Ketika client melakukan transaksi penulisan/perubahan data dalam kondisi jaringan normal, Node A menerima data tersebut dari gateway dan menampilkan log [NODE A] Saldo berhasil diperbarui secara lokal: 650000. Hal ini menandakan replikasi di sisi Node A sukses dilakukan.

- Output 2: python serverB.py

```
PS C:\Semester 6\SCALABLE SYSTEMS DESIGN\TUGAS_4\simulasiCAP> python serverB.py
[NODE B] Berjalan di port 5002...
[NODE B] Saldo berhasil diperbarui secara lokal: 650000
█
```

Terminal ini menjalankan skrip serverB.py yang bertindak sebagai Node B. Server ini berjalan secara independen di port 5002 untuk mendengarkan instruksi replikasi data.

Output menampilkan pesan inisialisasi awal [NODE B] Berjalan di port 5002....

Sama seperti Node A, ketika sistem melakukan sinkronisasi data saldo secara global, Node B mencetak log [NODE B] Saldo berhasil diperbarui secara lokal: 650000. Ini membuktikan bahwa mekanisme pengiriman pesan antar-node (message passing) berjalan dengan baik pada kondisi jaringan ideal.

- Output 3: python serverC.py

```
PS C:\Semester 6\SCALABLE SYSTEMS DESIGN\TUGAS_4\simulasiCAP> python serverC.py
[NODE C] Berjalan di port 5003...
[NODE C] Saldo berhasil diperbarui secara lokal: 650000
█
```

Terminal ini menjalankan skrip serverC.py yang bertindak sebagai Node C dan mendengarkan request pada port 5003. Node C ini dipersiapkan sebagai target yang akan diisolasi untuk menyimulasikan gangguan jaringan.

- Log awal menunjukkan [NODE C] Berjalan di port 5003....
- Pada transaksi pertama saat jaringan normal, Node C sukses menerima pembaruan data saldo menjadi 650000.
- Namun, setelah perintah pemutusan jaringan diaktifkan di gateway, terminal Node C berhenti menerima pembaruan data baru karena jalurnya telah terisolasi dari sisa klaster.

- Output 4: python server1.py

```
PS C:\Semester 6\SCALABLE SYSTEMS DESIGN\TUGAS_4\simulasiCAP> python server1.py
[SERVER1 - GATEWAY] Berjalan di port 5000...

=====
=== CONTROL PANEL SIMULASI CAP KELOMPOK 4 ===
Ketik 'putus' untuk mensimulasikan Jaringan Terputus (Partisi)
Ketik 'sambung' untuk menormalkan kembali jaringan
=====

[WRITE] Menerima request update saldo -> 650000 via Node_A...
[SYNC] Menyebarakan pembaruan ke Node_A, Node_B, dan Node_C...
█
```

Terminal ini menjalankan skrip server1.py yang berfungsi sebagai Pintu Gerbang Utama (Gateway) sekaligus pusat kendali (Control Panel) simulasi jaringan.

- Menampilkan pesan operasional [SERVER1 - GATEWAY] Berjalan di port 5000....

- Menampilkan menu interaktif kepada pengguna untuk merekayasa gangguan jaringan secara manual.
- Ketika administrator mengetikkan kata putus, terminal ini mencetak log peringatan:

Plaintext

[ERR] Jaringan terputus! Terjadi Network Partition pada Node_C!

[ALERT] Gerbang pengaman mendeteksi gangguan jaringan.

- Saat client mencoba menulis data baru ke sistem ketika partisi jaringan aktif, gateway akan langsung menolak operasi tersebut demi menjaga Konsistensi data (prinsip CP Theorem) agar tidak terjadi ketimpangan nilai saldo antar server.
- Output 5: python client.py

```
PS C:\Semester 6\SCALABLE SYSTEMS DESIGN\TUGAS_4\simulasiCAP> python client
.py
=== SIMULASI DISTRIBUTED SYSTEM KELOMPOK 4 ===
[INFO] Membuka koneksi ke Node_A, Node_B, Node_C...

--- 1. MEMBACA SALDO AWAL ---
[READ] Node_A saldo: 500000
[READ] Node_B saldo: 500000
[READ] Node_C saldo: 500000

--- 2. MENULIS DATA BARU (JARINGAN NORMAL) ---
Status Gateway: Replikasi berhasil disinkronkan ke seluruh node.
[READ] Node_A saldo: 650000
[READ] Node_B saldo: 650000
[READ] Node_C saldo: 650000

#####
PENTING: Sekarang, silakan pergi ke terminal SERVER1.PY
Ketik kata 'putus' lalu tekan Enter untuk membuat partisi jaringan.
#####

Jika sudah mengetik 'putus' di server1, tekan [ENTER] di sini untuk lanjut
simulasi penolakan...
```

Terminal ini menjalankan skrip client.py yang menyimulasikan perilaku pengguna (end-user) yang berinteraksi dengan sistem database terdistribusi melalui gateway.

- Fase 1 (Read Awal): Client sukses membaca saldo dari ketiga node yang bernilai sama, yaitu 500000.
- Fase 2 (Write Normal): Client berhasil mengubah saldo menjadi 650000 dan menerima notifikasi Replikasi berhasil disinkronkan ke seluruh node.
- Fase 3 (Write saat Partisi): Setelah koneksi Node C diputus, client mencoba melakukan penulisan data kembali. Karena klaster sedang tidak utuh, sistem mengaktifkan Consistency Guard dan mengirimkan pesan kegagalan ke terminal client: [ERR] Jaringan terputus ke Node_C! Mode Trade-off Aktif: Mengutamakan Konsistensi (CP). Transaksi ditolak.
>>> Status: Operation Failed (Consistency Guard Active)

3. Kesimpulan

Berdasarkan implementasi dan jalannya pengujian program simulasi sistem terdistribusi, diperoleh beberapa kesimpulan sebagai berikut:

- Validasi Teorema CAP: Simulasi berhasil mendemonstrasikan batasan Teorema CAP di dunia nyata. Ketika komponen jaringan mengalami gangguan (Partition), sistem harus memilih antara mengorbankan ketersediaan (Availability) demi menjaga akurasi data, atau sebaliknya.
- Keberhasilan Model Arsitektur CP: Program yang dibangun terbukti menerapkan karakteristik CP System (seperti pada basis data MongoDB atau sistem perbankan). Sistem memilih memblokir transaksi baru (mengorbankan Availability) daripada membiarkan data saldo antar-node menjadi tidak selaras atau korup akibat partisi jaringan.
- Fungsi Gateway Multi-Server: Penggunaan file `server1.py` sebagai koordinator sentral terbukti efektif dalam memantau kesehatan kluster (health check), menjembatani client request, serta melakukan simulasi rekayasa lalu lintas jaringan secara dinamis tanpa mengganggu integritas program utama di masing-masing node server.